The first two arguments to `mb_convert_variables()` are the source and target encoding, respectively, every subsequent argument is passed by reference and will be passed through the encoding conversion algorithm. Just like `mb_convert_encoding()`, you can pass arrays into `mb_convert_variables()`:

```
$data = [$determinist, $revisionist];
mb_convert_variables(
    'UTF-8',
    'UTF-16BE',
    $data
);
print bin2hex($data[0]) . PHP_EOL;
print bin2hex($data[1]);
```

The output will be identical to that of the previous example; note the array access in the `bin2hex()` call.

mb_scrub()

We've seen how character substitution works when converting encodings, but it is also possible to remove illegal byte sequences without performing an encoding transform. `mb_scrub()` will test the input string and substitute the sequence specified by the `subtitution_character` setting and retain the current encoding of the string. The input string is assumed to be in `internal_encoding` format.

```
// abc + broken surrogate pair
$brokenUtf16 = "\x00\x61\x00\x62\x00\x63\xD8\x00\xD8\x00";

mb_substitute_character(0xFFFD);
print bin2hex(mb_scrub($brokenUtf16)) . PHP_EOL;
print bin2hex(mb_scrub($brokenUtf16, 'UTF-16BE'));

Output:
006100620063efbfbd00efbfbd00
006100620063fffdfffd
```

The first call to `mb_scrub()` has no encoding argument. It is treated as UTF-8, which means that only the *0xD8* byte is treated as invalid (null bytes are fine). The UTF-8 encoding of U+FFFD is *0xEF 0xBF 0xBD*, and we can see that it is substituted into the